Team Progress Demo — 2026-05-12

What we shipped this trimester (since 1 March 2026)

Across the team, **24 pull requests have reached 2+ peer approvals** and are queued for the final upstream merge — covering input handling, graphics primitives, interface widgets, geometry, color, timers, and a generative-AI tutorial guide. Today's demo bundles **9 of those PRs** into a private staging branch (`demo/leadership-2026-05-12`) so the resulting site can be browsed live, end-to-end. A further 7 PRs sit at first-review; new contributions are still landing weekly.

How to read this demo

- All 9 features below are bundled on local branch **`demo/leadership-2026-05-12`** (none of these PRs are merged to upstream `main` yet — they're in the 2nd-review queue).
- Local preview: http://localhost:4321
- **Heads up on search:** the search box in the top bar will not return results in this local build. The DocSearch (Algolia) index is only populated against the production site at https://splashkit.io. If anyone asks to demo search, switch the browser tab to live and run the same query.
- Each feature below has a screenshot of how the function appears on the documentation page after the build pipeline auto-injects the C++ / C# (top-level) / C# (OOP) / Python tabs and the GIF / PNG output asset.

Feature walkthrough

1. `draw_circle` — graphics

URL: http://localhost:4321/api/graphics/#draw-circle

Draw Circle {</>}

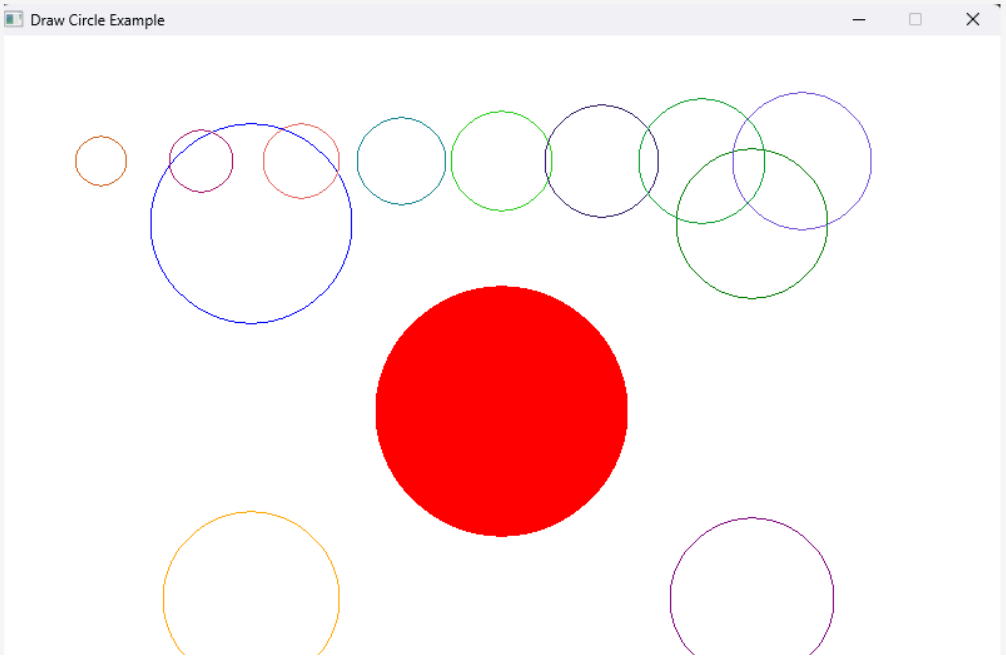
See Code Examples

Example 1: Circle Showcase

C++ | C# | Python

```cpp
#include "splashkit.h"

int main()
{
    open_window("Draw Circle Example", 800, 600);

    clear_screen(COLOR_WHITE);

    // Draw a large red filled circle in the center
    fill_circle(COLOR_RED, 400, 300, 100);

    // Draw circles with different colors, sizes, and positions
    draw_circle(COLOR_BLUE, 200, 150, 80);
    draw_circle(COLOR_GREEN, 600, 150, 60);
    draw_circle(COLOR_ORANGE, 200, 450, 70);
    draw_circle(COLOR_PURPLE, 600, 450, 65);

    // Draw smaller circles with varying colors
    for (int i = 0; i < 8; i++)
    {
        int radius = 20 + i * 5;
        int x = 400 + (i - 4) * 80;
        int y = 100;
        draw_circle(random_rgb_color(255), x, y, radius);
    }

    refresh_screen();

    delay(5000);
    close_all_windows();

    return 0;
}
```

Output:



Example 2: Animate a circle moving across the screen

C++ | Python

```cpp
#include "splashkit.h"

int main()
{
    open_window("Circle Animation", 800, 600);

    double x = 0;

    while (!quit_requested())
    {
        process_events();
        clear_screen(COLOR_WHITE);

        draw_circle(COLOR_RED, x, 300, 50);

        x += 2;
        if (x > 800) x = 0;

        refresh_screen(60);
    }

    return 0;
}
```

Output:

Example 3: Move a circle using arrow keys to demonstrate interactive usage of draw_circle.

C++ | Python

```cpp
#include "splashkit.h"

int main()
{
    open_window("Interactive Circle", 800, 600);

    double x = 400;
    double y = 300;

    while (!quit_requested())
    {
        process_events();
        clear_screen(COLOR_WHITE);

        if (key_down(LEFT_KEY)) x -= 5;
        if (key_down(RIGHT_KEY)) x += 5;
        if (key_down(UP_KEY)) y -= 5;
        if (key_down(DOWN_KEY)) y += 5;

        draw_circle(COLOR_BLUE, x, y, 50);

        refresh_screen(60);
    }

    return 0;
}
```

Output:

Adds two new variants for `draw_circle`: an animation example (a perpetually growing/shrinking circle) and an interactive example (radius controlled by mouse). Demonstrates the function on a real visual, not just signature documentation.

Author: @jankiluitel · PR #712

---

2. `key_down` — input

URL: http://localhost:4321/api/input/#key-down

Key Down {</>}

See Code Examples

Example 1: Keyboard State Display

C++ | C# | Python

```cpp
#include "splashkit.h"

void draw_key_status(string label, key_code key, double x, double y)
{
    bool pressed = key_down(key);

    color indicator;
    string state;

    if (pressed)
    {
        indicator = COLOR_GREEN;
        state = "Pressed";
    }
    else
    {
        indicator = COLOR_GRAY;
        state = "Released";
    }

    fill_circle(indicator, x, y, 25);
    draw_text(label + ": " + state, COLOR_BLACK, x + 40, y - 10);
}

int main()
{
    open_window("Keyboard State Display", 800, 400);

    while (!quit_requested())
    {
        process_events();

        clear_screen(COLOR_WHITE);

        draw_text("Press the arrow keys or space bar to see their current state.", COLOR_BLACK, 120, 40);

        draw_key_status("Left", LEFT_KEY, 120, 130);
        draw_key_status("Right", RIGHT_KEY, 120, 190);
        draw_key_status("Up", UP_KEY, 120, 250);
        draw_key_status("Down", DOWN_KEY, 120, 310);
        draw_key_status("Space", SPACE_KEY, 500, 220);

        refresh_screen(60);
    }

    return 0;
}
```

Output:

Detects whether a keyboard key is currently held down (vs `key_typed`, which fires once per press). Animated GIF shows the cursor changing colour as different keys are held.

Author: @jasveena15 · PR #702

---

3. `mouse_position` — input

URL: http://localhost:4321/api/input/#mouse-position

Mouse Position {</>}

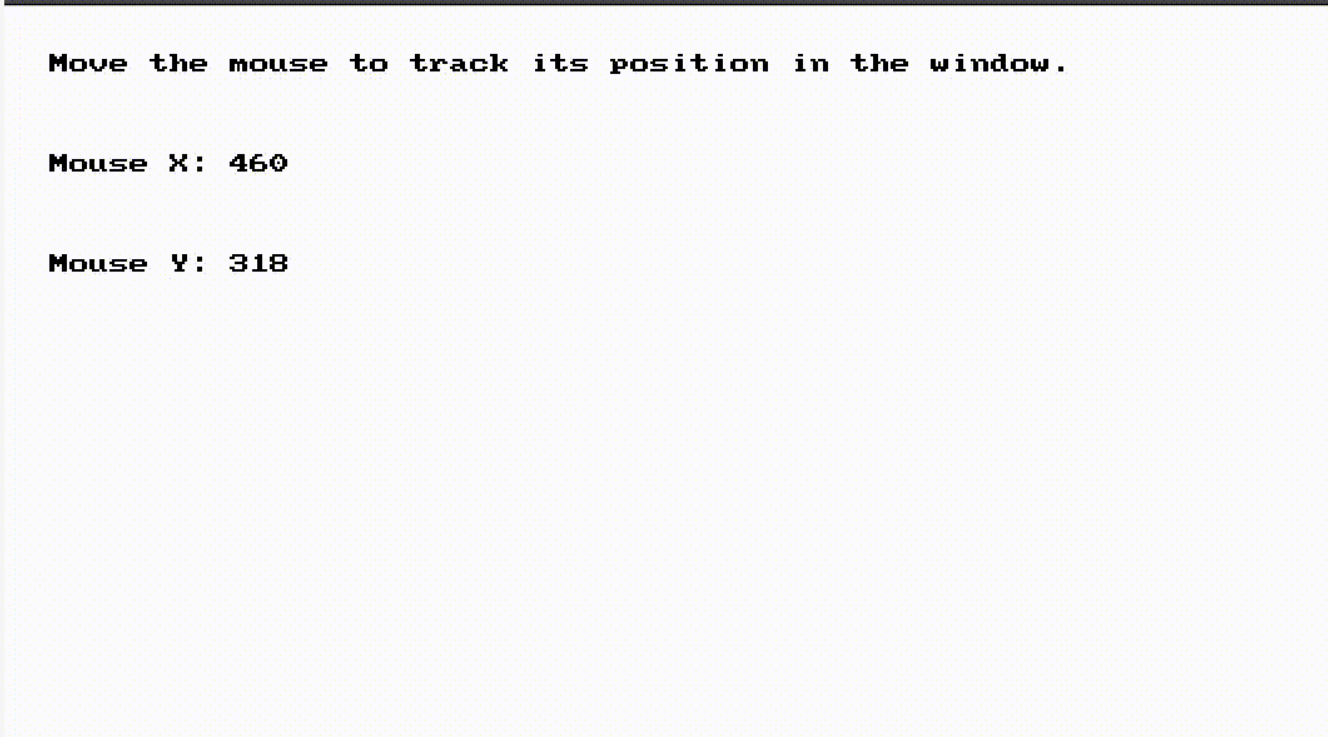
See Code Examples

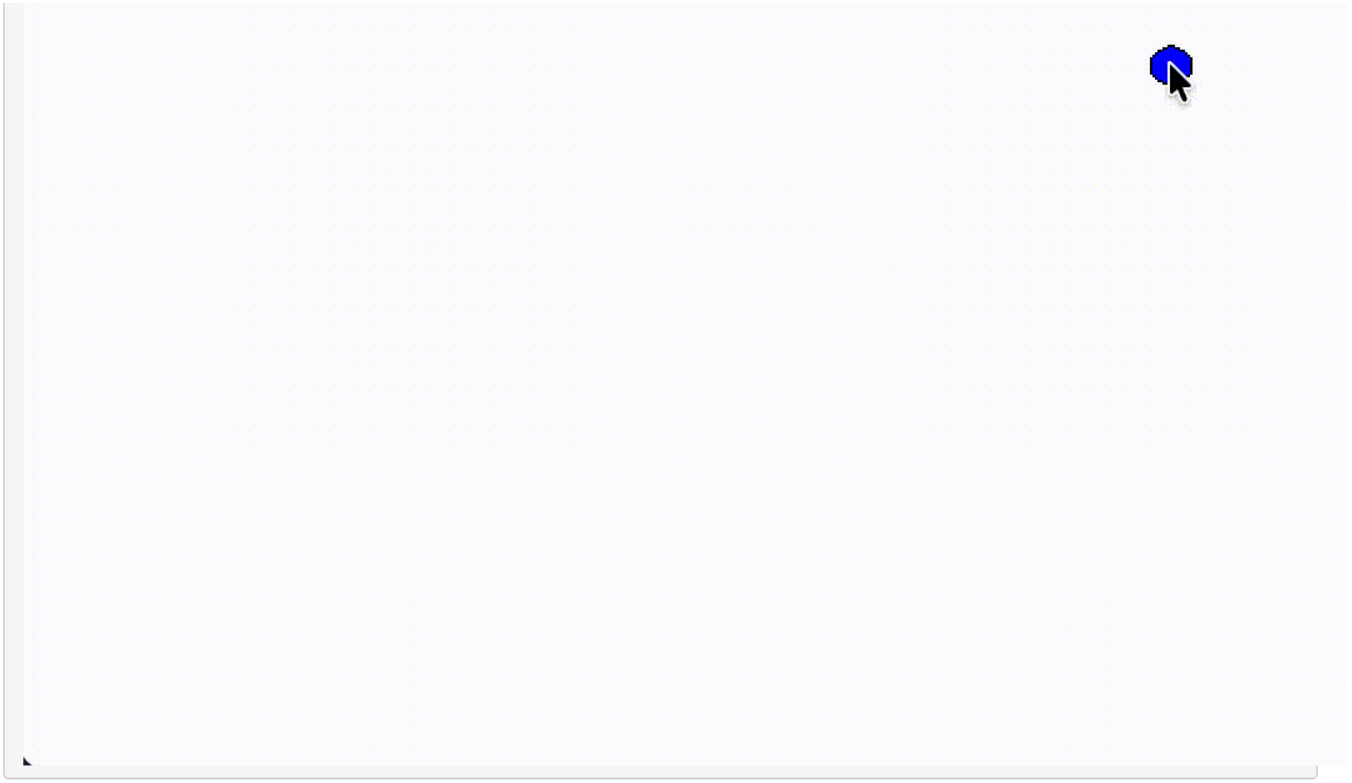
Example 1: Tracking Mouse Position in a Window

C++ | C# | Python

```cpp
#include "splashkit.h"

int main()
{
    open_window("Mouse Position Display", 800, 600);

    while (!quit_requested())
    {
        process_events();

        point_2d mouse_point = mouse_position();

        clear_screen(COLOR_WHITE);

        draw_text("Move the mouse to track its position in the window.", COLOR_BLACK, 20, 20);
        draw_text("Mouse X: " + to_string(mouse_point.x), COLOR_BLACK, 20, 60);
        draw_text("Mouse Y: " + to_string(mouse_point.y), COLOR_BLACK, 20, 100);

        fill_circle(COLOR_BLUE, mouse_point.x, mouse_point.y, 8);
        draw_circle(COLOR_BLACK, mouse_point.x, mouse_point.y, 8);

        refresh_screen(60);
    }

    close_all_windows();

    return 0;
}
```

Output:





Returns the cursor's current `(x, y)` as a `Point2D`. The example animates a small circle following the cursor in real time, with live coordinates rendered above.

Author: @Osaid2993 · PR #710

---

4. `key_typed` — input

URL: http://localhost:4321/api/input/#key-typed

Key Typed {</>}

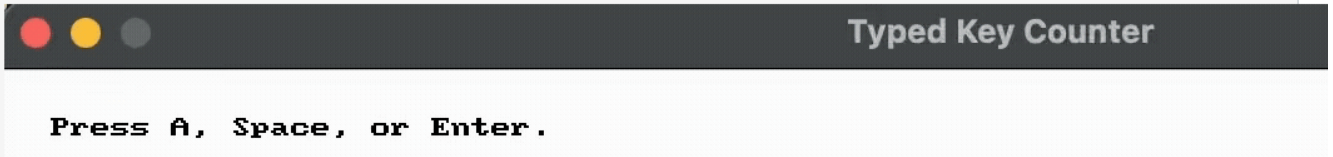
See Code Examples

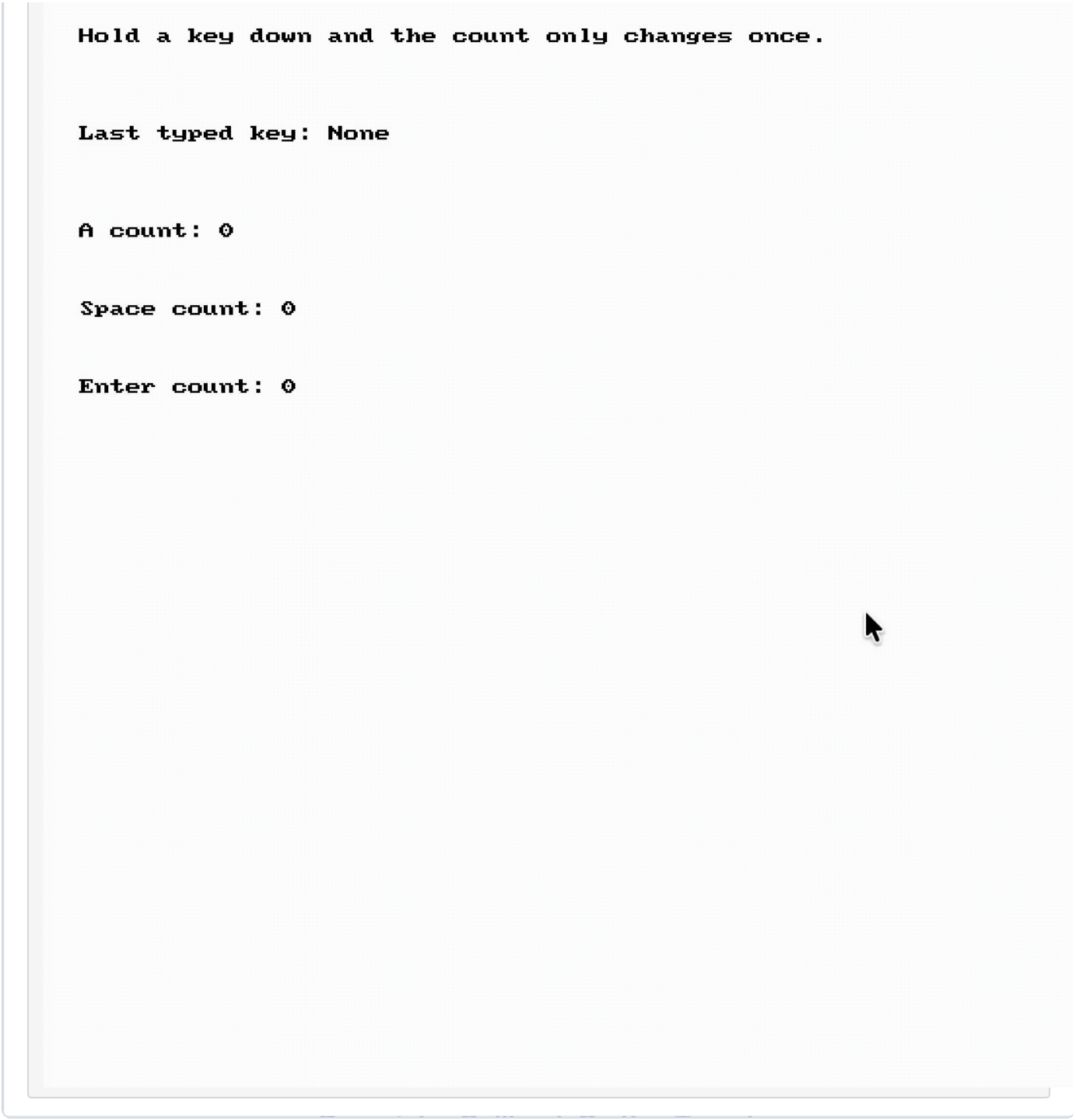
Example 1: Typing Keys Once with KeyTyped

C++ | C# | Python

```cpp
#include "splashkit.h"
#include <string>

int main()
{
    open_window("Typed Key Counter", 800, 600);

    int a_count = 0;
    int space_count = 0;
    int enter_count = 0;
    std::string last_typed_key = "None";

    while (!quit_requested())
    {
        process_events();

        // Count each key once when it is first pressed.
        if (key_typed(A_KEY))
        {
            a_count++;
            last_typed_key = "A";
        }

        if (key_typed(SPACE_KEY))
        {
            space_count++;
            last_typed_key = "Space";
        }

        if (key_typed(RETURN_KEY))
        {
            enter_count++;
            last_typed_key = "Enter";
        }

        clear_screen(COLOR_WHITE);

        draw_text("Press A, Space, or Enter.", COLOR_BLACK, 20, 20);
        draw_text("Hold a key down and the count only changes once.", COLOR_BLACK, 20, 50);
        draw_text("Last typed key: " + last_typed_key, COLOR_BLACK, 20, 100);
        draw_text("A count: " + std::to_string(a_count), COLOR_BLACK, 20, 150);
        draw_text("Space count: " + std::to_string(space_count), COLOR_BLACK, 20, 190);
        draw_text("Enter count: " + std::to_string(enter_count), COLOR_BLACK, 20, 230);

        refresh_screen(60);
    }

    return 0;
}
```

Output:





The "single press" sibling of `key_down`. Fires exactly once per key press, ideal for menu navigation or toggle behaviour. The GIF demonstrates one event per tap.

Author: @Osaid2993 · PR #732

---

5. `key_released` — input

URL: http://localhost:4321/api/input/#key-released

Key Released {</>}

See Code Examples

Example 1: Key Release Counter

C++ | C# | Python

```cpp
#include "splashkit.h"

int main()
{
    open_window("Key Released", 800, 600);

    int release_count = 0;
    std::string status = "Waiting...";

    while (!quit_requested())
    {
        process_events();

        if (key_down(SPACE_KEY))
        {
            status = "Holding Space...";
        }

        // key_released returns true only once per release event
        if (key_released(SPACE_KEY))
        {
            release_count++;
            status = "Space released!";
        }

        clear_screen(COLOR_WHITE);
        draw_text("Press and hold [SPACE], then release it", COLOR_BLACK, "Arial", 18, 200, 220);
        draw_text("Status: " + status, COLOR_DARK_GRAY, "Arial", 18, 200, 270);
        draw_text("Times released: " + std::to_string(release_count), COLOR_BLUE, "Arial", 24, 200, 320);
        refresh_screen(60);
    }

    close_all_windows();
    return 0;
}
```

Output:

Fires when a previously-held key is let go — useful for charge-up mechanics, hold-to-aim controls, etc. Closes out the keyboard event-state trio with `key_down` and `key_typed`.

Author: @abdullah12121212 · PR #733

---

6. `mouse_shown` — input

URL: http://localhost:4321/api/input/#mouse-shown

Mouse Shown {</>}

See Code Examples

Example 1: Mouse Visibility Toggle

C++ | C# | Python

```cpp
#include "splashkit.h"

int main()
{
    open_window("Mouse Shown", 800, 600);

    while (!quit_requested())
    {
        process_events();

        // Press S to show the mouse, H to hide it
        if (key_typed(S_KEY))
            show_mouse();

        if (key_typed(H_KEY))
            hide_mouse();

        // mouse_shown returns true if the mouse cursor is currently visible
        string status;
        if (mouse_shown())
            status = "Mouse is visible";
        else
            status = "Mouse is hidden";

        clear_screen(COLOR_WHITE);
        draw_text("Press S to show the mouse, H to hide it", COLOR_BLACK, "Arial", 18, 170, 250);
        draw_text(status, COLOR_BLUE, "Arial", 24, 170, 290);
        refresh_screen(60);
    }

    // Always show the mouse again before closing
    show_mouse();
    close_all_windows();
    return 0;
}
```

Output:

Bonus example bundled in the same PR as `key_released`. Returns whether the OS cursor is currently visible in the SplashKit window — pairs with `hide_mouse` / `show_mouse` for cursor-locked games.

Author: @abdullah12121212 · PR #733

---

7. `any_key_pressed` — input

URL: http://localhost:4321/api/input/#any-key-pressed

Any Key Pressed {</>}

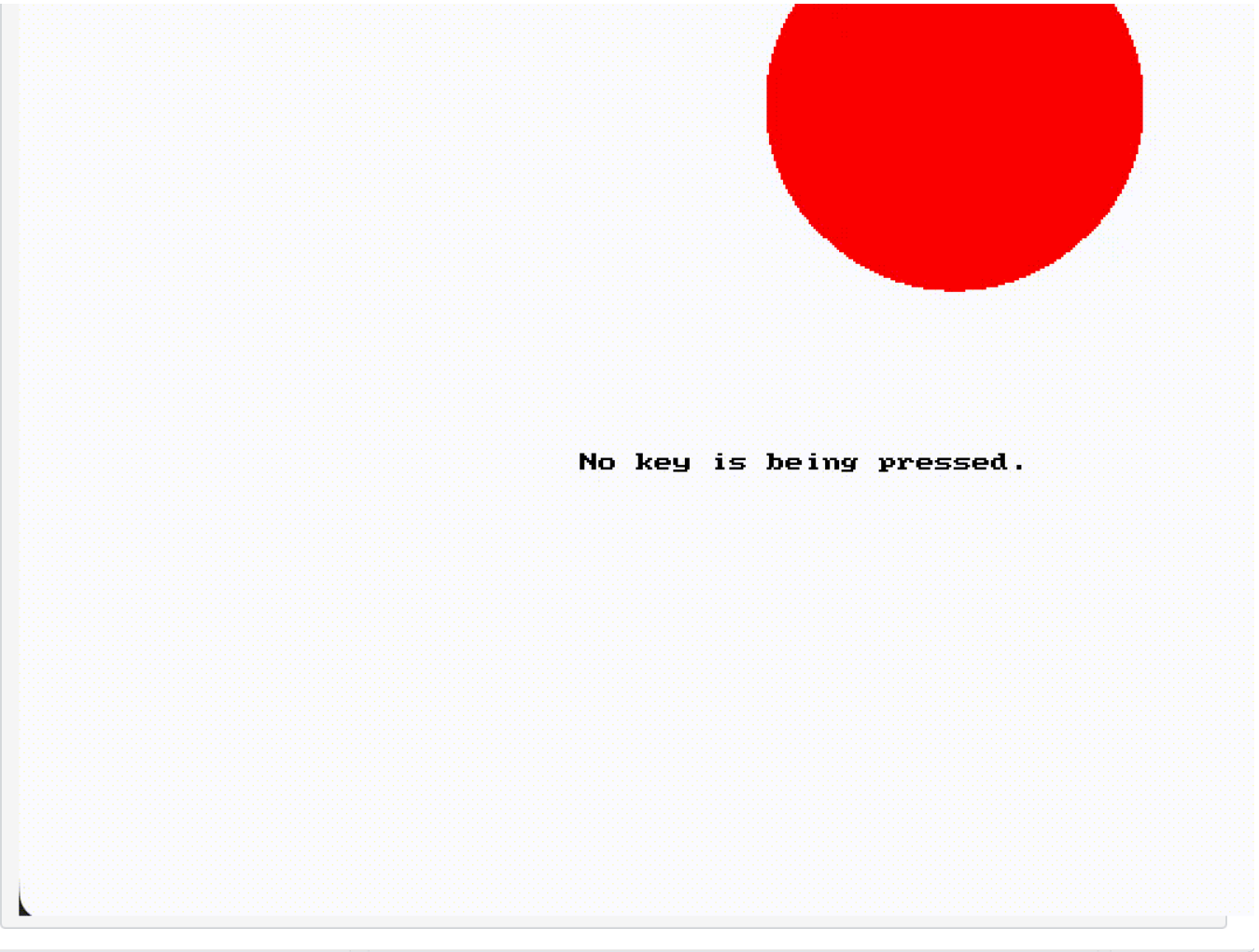
See Code Examples

Example 1: Any Key Press Indicator

C++ | C# | Python

```cpp
#include "splashkit.h"

int main()
{
    open_window("Any Key Pressed", 800, 600);

    std::string message = "No key is being pressed.";
    color circle_color = COLOR_RED;

    while (!quit_requested())
    {
        process_events();

        if (any_key_pressed())
        {
            message = "A key is being pressed.";
            circle_color = COLOR_GREEN;
        }
        else
        {
            message = "No key is being pressed.";
            circle_color = COLOR_RED;
        }

        clear_screen(COLOR_WHITE);

        fill_circle(circle_color, 400, 250, 80);
        draw_text(message, COLOR_BLACK, 240, 400);

        refresh_screen(60);
    }

    close_all_windows();
    return 0;
}
```

Output:



Convenience function for "press any key to continue" splash screens — fires `true` when any keyboard key is currently down without needing to enumerate `KeyCode` values.

Author: @Osaid2993 · PR #734

---

8. `key_name` — input

URL: http://localhost:4321/api/input/#key-name

Key Name {</>}

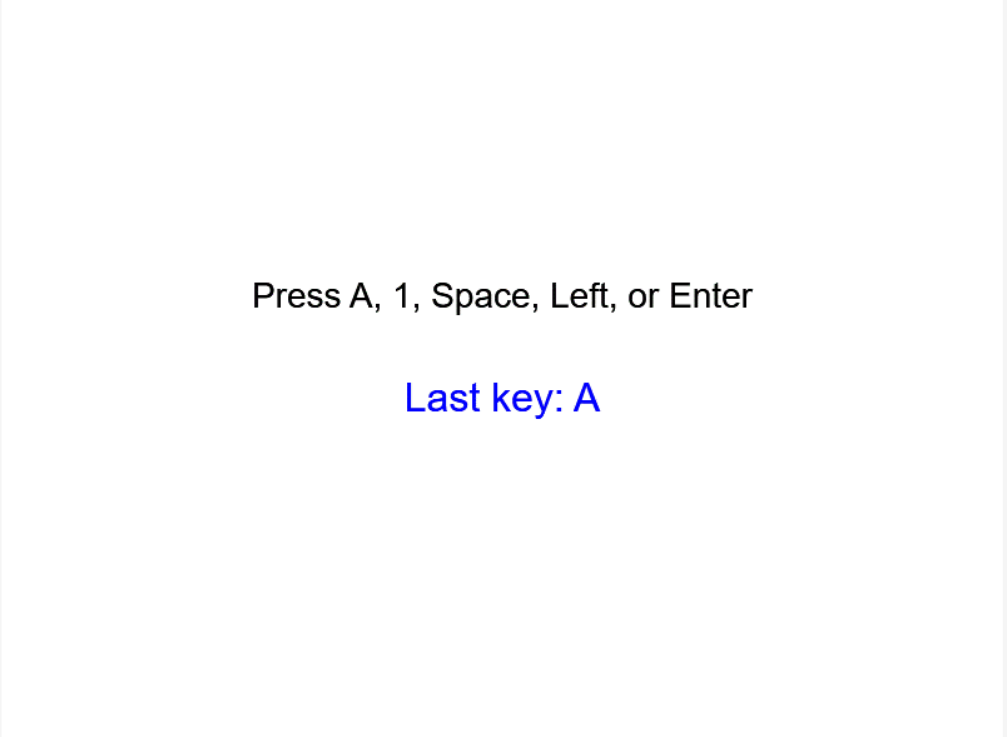
See Code Examples

Example 1: Show Pressed Key

C++ | C# | Python

```cpp
#include "splashkit.h"

int main()
{
    open_window("Key Name", 800, 600);

    // Store the last key typed from this example's set of demo keys
    key_code last_key = UNKNOWN_KEY;

    while (!quit_requested())
    {
        process_events();

        // Check which demo key was typed and save its key code
        if (key_typed(A_KEY)) last_key = A_KEY;
        if (key_typed(NUM_1_KEY)) last_key = NUM_1_KEY;
        if (key_typed(SPACE_KEY)) last_key = SPACE_KEY;
        if (key_typed(LEFT_KEY)) last_key = LEFT_KEY;
        if (key_typed(RETURN_KEY)) last_key = RETURN_KEY;

        // Draw the instructions and the readable name of the last key
        clear_screen(COLOR_WHITE);
        draw_text("Press A, 1, Space, Left, or Enter", COLOR_BLACK, 150, 220);
        draw_text("Last key: " + key_name(last_key), COLOR_BLUE, 280, 300);
        refresh_screen();
    }

    close_all_windows();
    return 0;
}
```

Output:



Returns the human-readable name for a `KeyCode` (e.g. `"Space"`, `"Left Arrow"`). Useful for in-game key-binding UIs that show players what's mapped where.

Author: @KayDee6751 · PR #745

---

9. `button` — interface

URL: http://localhost:4321/api/interface/#button

Button {</>}

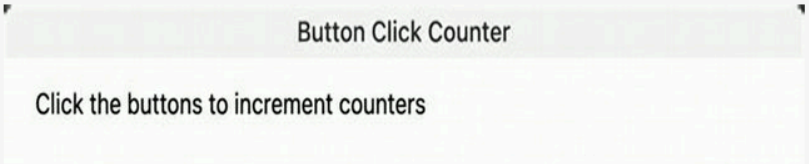
See Code Examples

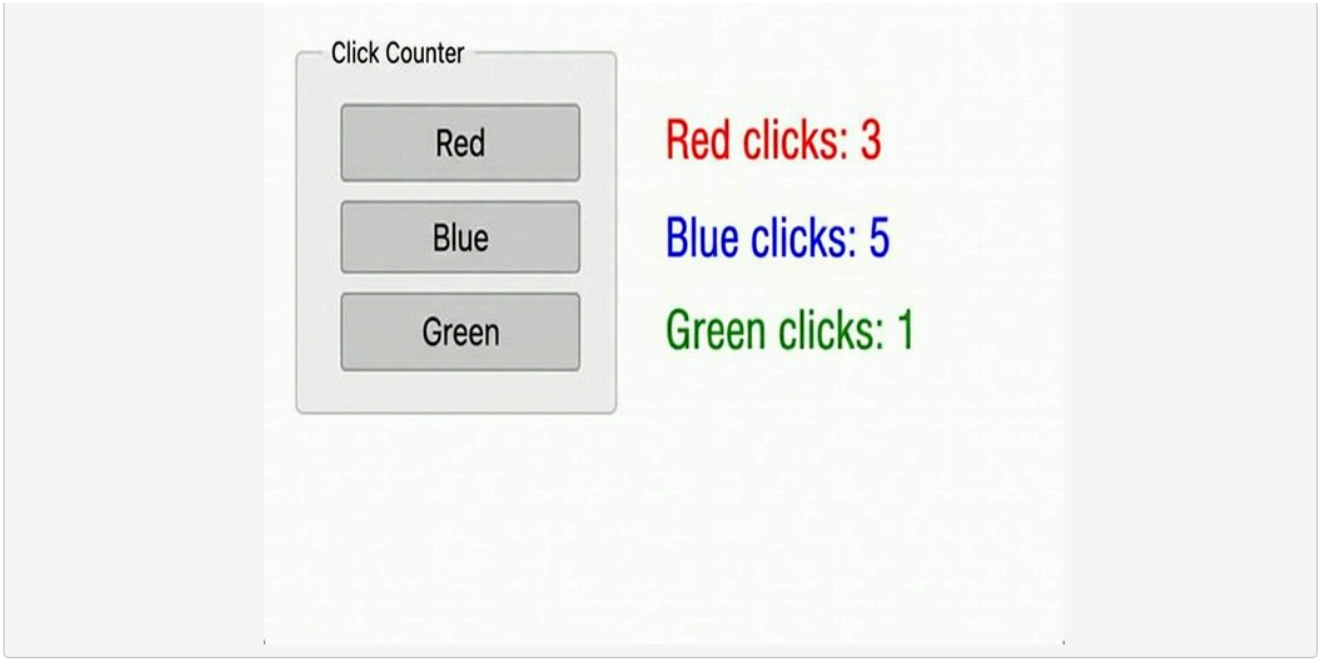
Example 1: Button Click Counter

C++ | C# | Python

```cpp
#include "splashkit.h"

int main()
{
    open_window("Button Click Counter", 800, 600);

    // Track click counts for each button
    int red_count = 0;
    int blue_count = 0;
    int green_count = 0;

    while (!quit_requested())
    {
        process_events();
        clear_screen(COLOR_WHITE);

        // Show instructions for the user
        draw_text("Click the buttons to increment counters", COLOR_BLACK, 10, 10);

        // Create a panel with three buttons
        if (start_panel("Click Counter", rectangle_from(50, 50, 200, 200)))
        {
            // Check if each button is clicked and update counts
            if (button("Red"))
            {
                red_count++;
            }
            if (button("Blue"))
            {
                blue_count++;
            }
            if (button("Green"))
            {
                green_count++;
            }
            end_panel("Click Counter");
        }

        // Display each button's click count on the window
        draw_text("Red clicks: " + std::to_string(red_count), COLOR_RED, 300, 100);
        draw_text("Blue clicks: " + std::to_string(blue_count), COLOR_BLUE, 300, 130);
        draw_text("Green clicks: " + std::to_string(green_count), COLOR_GREEN, 300, 160);

        refresh_screen(60);
    }

    close_all_windows();
    return 0;
}
```

Output:





A first-class interface widget — renders a clickable button with hover/active states and returns `true` on click. Shifts SplashKit's UI offering from "draw your own" to "compose with provided primitives."

Author: @ralphweng-autograb · PR #709

---

10. AI Prompt Helper for SplashKit Game Ideas — generative-AI guide

URL: http://localhost:4321/guides/generative-ai/ai-prompt-helper-for-splashkit/

AI Prompt Helper for SplashKit Game Ideas

Learn how to use AI tools responsibly to generate game ideas, debug code, and understand SplashKit concepts.

Written by: Janki Luitel

Last updated: 2026-05-03T00:00:00.000Z

Artificial Intelligence (AI) tools can support your learning journey when building games with SplashKit. AI should be used as a learning assistant — not as a replacement for your own problem-solving, coding, and creativity.

Why Use AI with SplashKit?

AI tools can help you:

- Brainstorm game ideas
- Break complex programming tasks into smaller steps
- Understand SplashKit functions
- Debug code errors
- Generate pseudocode before implementation
- Learn programming concepts faster

Example Prompt 1: Game Idea Generation

> "Give me 5 beginner game ideas using SplashKit in C++."

Example outputs may include:

- Maze Escape Game
- Space Shooter

- Space Shooter
- Flappy Bird Clone
- Bouncing Ball Physics Game
- Platform Adventure

Example Prompt 2: Understanding SplashKit Functions

> "Explain how draw_circle works in SplashKit with a simple example."

AI can help explain:

- Parameters
- Colours
- Coordinates
- Radius
- Real-world use cases

Example Prompt 3: Debugging

> "Why am I getting includePath errors in SplashKit C++?"

AI may help identify:

- Missing SDK paths
- Incorrect compiler settings
- VS Code configuration issues

Example Prompt 4: Pseudocode Planning

> "Help me design pseudocode for a SplashKit bouncing ball game."

This can help you plan:

- Variables
- Movement logic
- Collision detection
- Game loop structure

Bad Prompt vs Good Prompt

Bad prompt:

> "Fix my SplashKit code."

Good prompt:

> "I am using SplashKit in C++. My goal is to draw a circle in a window, but the window closes immediately. Explain why this happens and suggest how to fix it without rewriting the whole program."

Responsible Use of AI

When using AI:

✅ Use AI to learn concepts
✅ Ask AI for explanations
✅ Use AI for brainstorming
✅ Verify AI-generated code

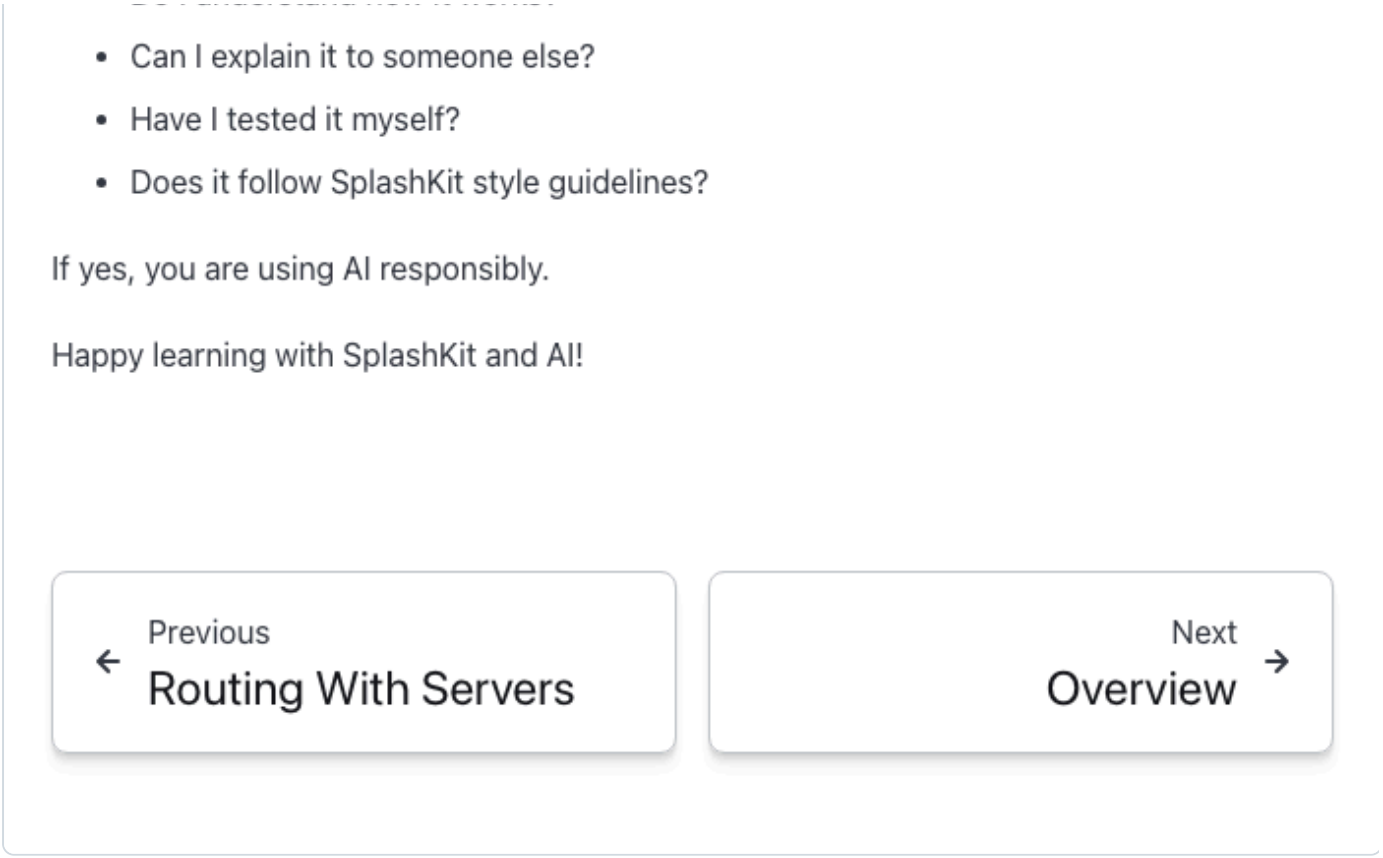
Avoid:

❌ Copying code without understanding it
❌ Submitting AI-generated work as your own
❌ Relying on AI without testing your code

Final Checklist

Before using AI-generated ideas or code:

- Do I understand how it works?



A different artifact type — a tutorial guide page (not a usage example). Walks SplashKit learners through using AI tools responsibly: prompting patterns for game-idea generation, function lookup, debugging, and pseudocode planning, plus a "good prompt vs bad prompt" comparison and a responsible-use checklist.

Author: @jankiluitel · PR #749

Anticipated questions

"Why aren't these merged to upstream `main` yet?" They've passed peer review (2+ approvals each) and are in the maintainer queue at `thoth-tech/splashkit.io-starlight`. This demo branch bundles them so the team's progress can be evaluated as a coherent slice rather than waiting for the upstream merge cadence.

"How are usage examples authored?" Each example is six files under `public/usage-examples/<category>/`: `.cpp`, `.cs` (top-level), `.cs` (OOP), `.py`, a one-line `.txt` title, and a `.gif` / `.png` / `.webm` output asset. The build pipeline (`scripts/api-pages-script.cjs`) scans these at `npm run build` time and auto-injects them into the corresponding API reference MDX page as a tabbed code block plus the visual.

"What's the test coverage story?" Code samples are compile-checked at build time via `usage-examples-testing-script.cjs` to catch broken samples before they reach the published docs. The build also gates on Astro's content-collection schema validation.

"Why is local search disabled?" The site uses Algolia DocSearch, which crawls the production deployment to build its index. Local builds intentionally don't ship that index — only the live `splashkit.io` site has searchable content. If you want to demo search, jump to live.

"How many PRs are in flight overall?" 24 PRs have 2+ approvals and are ready to merge upstream. 7 more sit at first-review. 12 are in the second-review queue. Today's demo shows 9 of the 24 ready-to-merge tier.

What's next

- More usage-example coverage across remaining API surface (geometry overloads, networking, audio).
- Continue the second-review queue clear-out so the upstream `main` branch reflects current team output.
- Expand the generative-AI guide stream — author handles a tutorial pattern other contributors can follow.

This document corresponds to local branch `demo/leadership-2026-05-12` built at 2026-05-12. Backup at https://github.com/ralphweng2023/splashkit.io-starlight/tree/demo/leadership-2026-05-12 after push.